

MUSC 320 -- Week 7: Introduction to MSP



Welcome to the MSP module of MUSC 320. This repository contains everything you need for the lecture: Max patches to open and explore, a quick reference for the MSP objects we cover, and signal-flow diagrams.

Getting started

You will need **Max 9** installed to open the patches.

Download the entire repo

Click the green **Code** button at the top of this page, then choose **Download ZIP**. Unzip the folder and open any `.maxpat` file in Max.

If you are comfortable with Git you can also clone:

```
git clone https://github.com/taylorbrook/MUSC320_signals.git
```

Key Concepts

- **Signal flow:** Source -> Processing -> Output (always).
- **Bridge objects:** sig~ connects the Max world to the MSP world.
- **Amplitude matters:** Always scale signals before output. Full-amplitude noise~ directly to dac~ = dangerously loud.
- **The output chain:** your signal -> live.gain~ -> clip~ -> dac~ (always use this pattern).
- **DSP must be on:** Toggle ezdac~ or check Audio Status before expecting sound.

Assignment

- [Create a Sound Sculpture](#) -- Build your own sound sculpture using the MSP objects from class

Patches

Open these in order. Each patch builds on the previous one, leading up to a full noise-sculpture performance instrument.

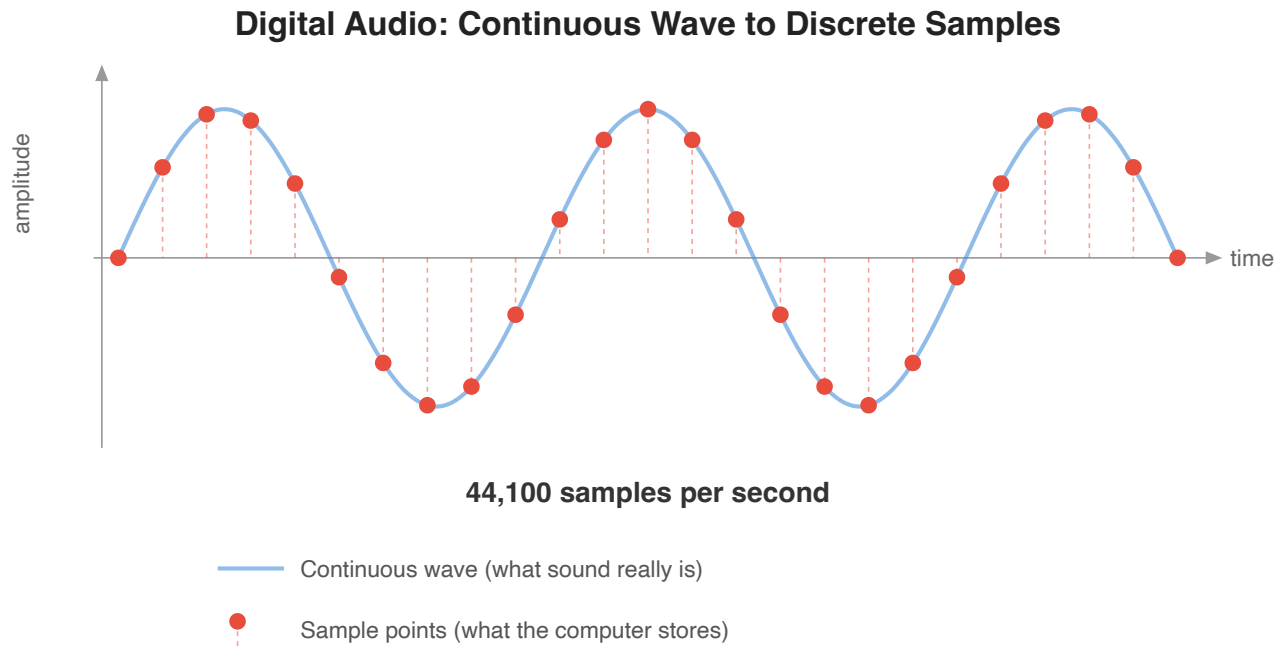
#	Patch	What it covers
1	01-control-vs-signals-three-level-comparison.maxpat	Control rate vs. signal rate vs. sample-level processing side by side
2	02-first-tone.maxpat	Controlling frequency and amplitude with <code>cycle~</code> and <code>*~</code>
3	03-control-meets-signal.maxpat	Bridging Max messages and MSP signals with <code>sig~</code> and <code>number~</code>
4	04-noise-and-amplitude.maxpat	White noise, <code>noise~</code> , and shaping amplitude
5	05-envelopes.maxpat	Amplitude envelopes with <code>line~</code>
6	06-modulation-and-mixing.maxpat	LFO modulation, AM/ring mod, and mixing voices with <code>~+</code>
7	07-noise-sculpture-rebuild.maxpat	Rebuilding the demo from scratch -- putting it all together

Bonus

- [noise-sculpture-demo.maxpat](#) -- The finished performance instrument demonstrated in lecture

What Is a Signal?

MSP objects process **signals** -- continuous streams of numbers, 48,000 or 44,100 per second. Every MSP object has a **tilde (~)** in its name. When you see ~, think "audio rate."



In MSP: `adc~` converts analog input to digital samples — `dac~` converts digital samples back to analog output

Signals vs. messages: A Max message (bang, int, float) arrives when triggered. A signal flows continuously -- 48,000 or 44,100 values every second, whether you ask for them or not.

MSP Object Reference

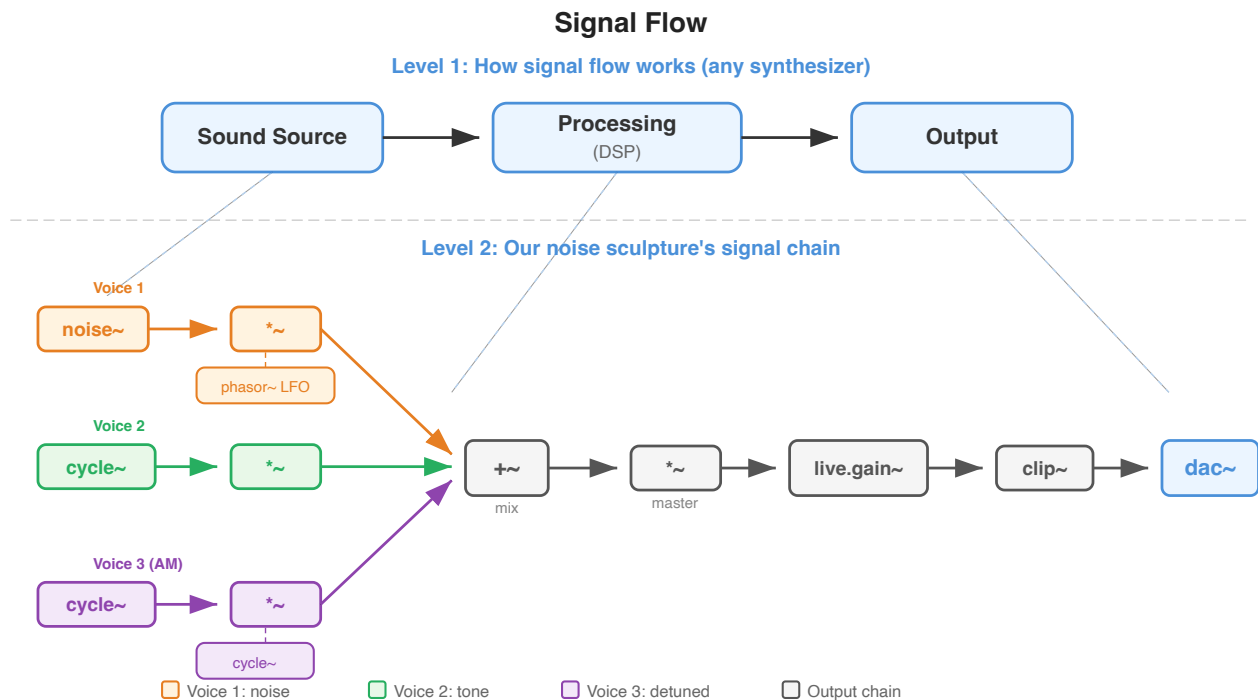
Signal Sources

Objects that generate signals from nothing.

- `cycle~` -- cosine/sine oscillator. First argument: frequency in Hz. Connect a signal to the left inlet for frequency modulation. ("metro generates bangs at intervals; cycle~ generates a smooth wave")
- `noise~` -- white noise generator. No arguments. Outputs random values -1 to 1 at audio rate. ("random gives one random number per bang; noise~ gives one every sample")
- `phasor~` -- repeating 0-to-1 ramp. First argument: frequency in Hz. Used as an LFO or automation source. ("metro fires bangs at intervals; phasor~ generates a smooth repeating ramp")

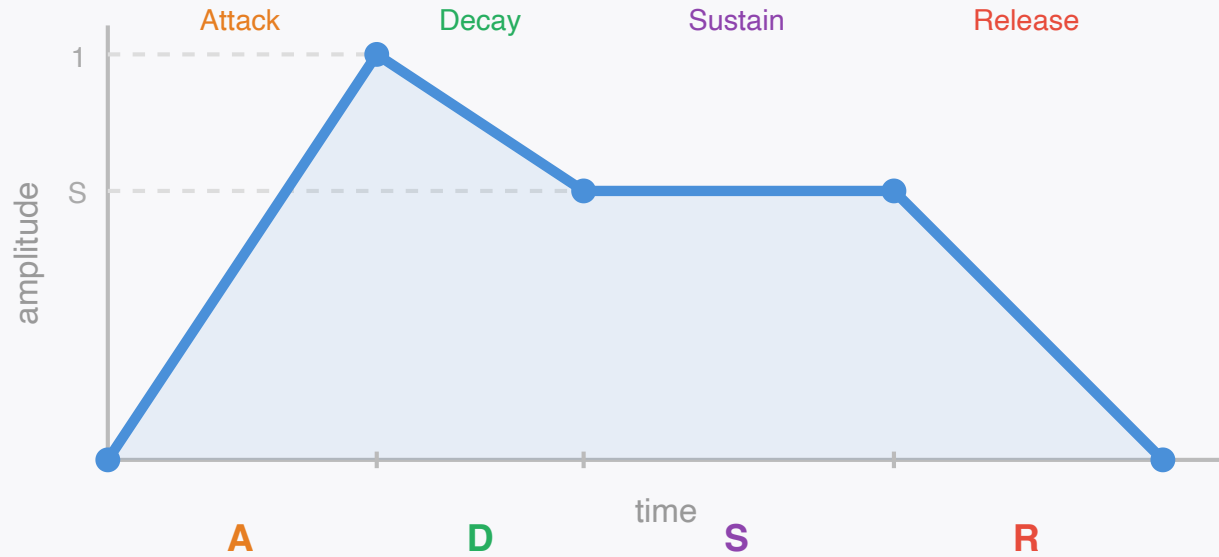
Signal Operators

Objects that transform signals.



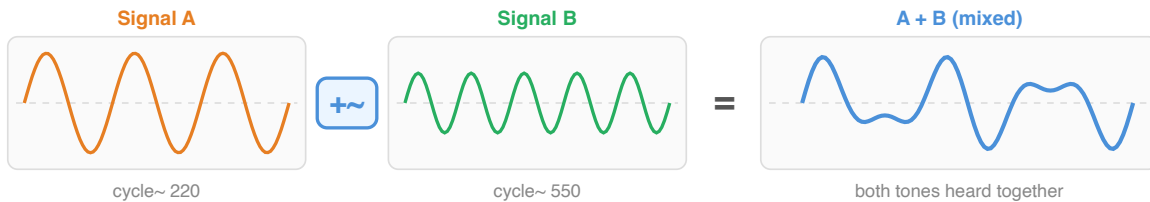
- `*~` -- multiplies two signals, or a signal by a constant. Used for amplitude control, ring modulation, AM synthesis. ("You used `*` to multiply numbers; `*~` multiplies signals")
- `+~` -- adds two signals. Used for mixing voices or adding DC offset. ("You added numbers with `+`; `+~` adds signals")
- `sig~` -- converts a Max number to a continuous signal. The bridge between Max and MSP.
- `line~` -- generates smooth ramps at audio rate. Message format: `target time` (e.g., `1. 500` ramps to 1.0 over 500 ms). ("line~ is the signal version of line -- smooth ramps at audio rate instead of message rate")

ADSR Envelope



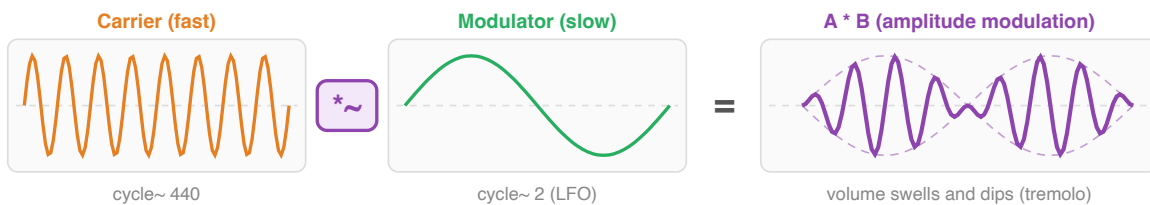
Adding and Multiplying Signals

+~ adds two signals together (mixing)



Adding signals combines them into one sound. This is how you mix voices in a patch.

*~ multiplies two signals together (amplitude control)



Multiplying shapes one signal's amplitude with another. This is how tremolo, AM, and ring mod work.

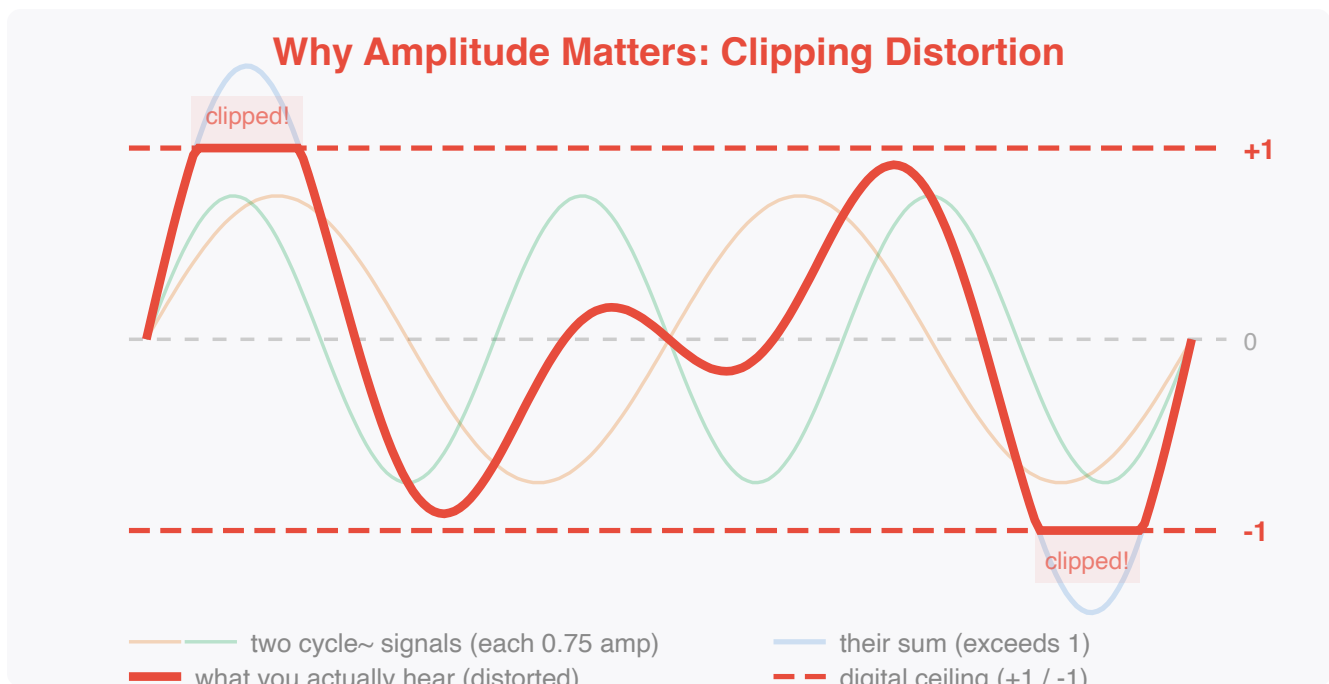
— Input signal — Input signal — +~ result — *~ result - - - envelope

Output and Safety

Getting signal to speakers safely.

- `dac~` -- digital-to-analog converter. Sends signal to speakers. Two inlets = left/right stereo.

- `ezdac~` -- same as `dac~` but with a click-to-toggle UI. The power switch for MSP audio.
- `live.gain~` -- volume control with built-in metering. Stereo in/out. Default to -12 dB or lower.
- `clip~` -- limits signal to a range. `clip~ -0.9 0.9` prevents dangerously loud output. Safety net before `dac~`.



Monitoring

Seeing signals.

- `live.scope~` -- oscilloscope display. Shows waveform shape in real time.
- `number~` -- displays signal value at control rate. Mode 2 = display only (no output).

Bonus

Objects worth exploring on your own -- option-click any object in Max to open its help file.

- `pink~` -- pink noise generator. Like `noise~` but with less high-frequency energy, producing a warmer, more natural noise.
- `onepole~` -- simple one-pole low-pass filter. Smooths a signal by rolling off high frequencies. Useful for shaping noise or smoothing control signals.
- `adsr~` -- attack-decay-sustain-release envelope generator. A more flexible alternative to building envelopes with `line~` -- one object handles the full ADSR shape.